Computer Games Development CW208
Technical Design Document
Year IV

Olawole Abayomi-Owodunni
C00278711

Olawole Abayomi-Owodunni
C00278711

13/04/2026

[Declaration form to be attached]

Contents

Overview 4
Key Terminology 4
Goals and Technical Requirements 5
Must-Have Goals (Definition of Done) 5
Platform, Tooling and Constraints 5
External Dependencies 5
Scope and Non-Goals 6
In Scope (Current / Near-Term) 6
Out of Scope 6
Intentional Simplifications 6
Software Architecture 7
Architectural Style and Rationale 7
High-Level System Diagram 7
Major Subsystem Overview 8
Ownership and Data Flow 8
Class Design and Specifications 10
Core Class Hierarchy (UML Class Diagram — Textual Representation) 10
ICollidable Interface 11
Enemy Hierarchy 11
Steering Behaviours (Strategy Pattern) 11
Projectile Hierarchy 11
Weapon System 12
Room Generation Pipeline 12
CRC Cards 12
CRC Card: Game 12
CRC Card: Player 13
CRC Card: Enemy (abstract) 13
CRC Card: GruntEnemy 13
CRC Card: TurretEnemy 13

CRC Card: FloorGenerator 13
CRC Card: RoomInstance 14
CRC Card: LLMService 14
CRC Card: QuestManager 14
CRC Card: CollisionCheck 14
Algorithm Design 15
Floor Graph Generation Algorithm 15
Floor Layout Embedding Algorithm 15
Corridor Generation Algorithm 16
Collision Detection and Resolution 17
Combat Wave Management 18
LLM Asynchronous Job Queue 18
Runtime Flow 20
Per-Frame Update Sequence 20
State Transitions (Game Modes) 21
Data and State Management 22
Runtime State Ownership 22
Procedural Generation State 22
Critical Analysis 23
Known Prototype Shortcuts 23
Technical Risks 24
Evaluation Against Goals 24
Conclusions 26
References 27
Books 27
Academic Papers and Reports 27
Web Resources 27

Overview

Key Terminology

Term	Definition
SFML	Simple and Fast Multimedia Library - a cross-platform C++ library providing windowing, graphics, audio, and input functionality.
Roguelike	A subgenre of role-playing games characterised by procedurally generated levels, permadeath, and turn-based or real-time combat.
Hub World	A persistent safe area the player returns to between dungeon runs, used for upgrading and progression.
Dungeon -> Floor -> Room	The hierarchical procedural generation structure: a dungeon consists of multiple floors, each floor is a graph of interconnected rooms.
Layer / Mask Collision Filtering	"Layer" defines what an object is (e.g. PLAYER_LAYER); "Mask" defines what it can collide with. Two objects collide only if each object's layer is present in the other's mask.
ICollidable	A C++ interface (abstract class) that any object participating in collision checks must implement, providing getCollisionBounds() and getCollisionProfile().
RoomPlan	A lightweight data-only blueprint describing a room's tile layout, doors, spawners, and triggers — no SFML objects.
RoomInstance	The runtime representation built from a RoomPlan, containing renderable SFML shapes and static collision objects.
Seeded Procedural Generation	A technique using a deterministic random number generator (std::mt19937) seeded with a known value so that the same seed always produces the same output.
LLM (Large Language Model)	A machine learning model trained on large text corpora, capable of generating human-like text. In this project, the model runs locally via llama.cpp.
Steering Behaviours	Autonomous agent movement algorithms (Seek, Arrive, Flee, etc.) that produce acceleration vectors based on agent state and environment.
AABB	Axis-Aligned Bounding Box - a rectangular collision volume whose sides are parallel to the coordinate axes.

Goals and Technical Requirements

Must-Have Goals (Definition of Done)

The following goals represent the minimum viable product that must be achieved for the project to be considered complete:

- Playable core loop: The player can move, aim, and attack. Enemies spawn and behave consistently. Collisions and damage feel reliable.
- Procedural dungeon structure: Generate Floors → Rooms from a seed. Support core room types (spawn, combat, portal) at minimum.
- Upgrade loop: A hub world exists where currency is earned during runs and the player can spend it on upgrades.
- NPC interaction: NPCs can be interacted with. Dialogue can be driven by a local LLM integration.

Platform, Tooling and Constraints

Requirement	Detail
Language	C++ (C++17 standard or later)
Graphics / Input	SFML 3.x
IDE / Build	Visual Studio (MSBuild .sln/.vcxproj)
OS Target	Windows (x64 primary; x86 also tested)
Controller Input	XInput via a custom Gamepad wrapper class
Game Loop	Fixed timestep (~60 FPS); dt passed in seconds
Collision System	Bitmask layer/mask filtering with AABB intersection
Procedural Gen (Seeded)	std::mt19937 seeded with dungeonSeed + floorId for deterministic floor graphs
Procedural Gen (Random)	rand() for per-run room detail variety (obstacle placement, spawns)

External Dependencies

The project relies on the following external dependencies:

- **SFML 3.x:** Windowing, 2D rendering, input handling, and audio (linked as a static/dynamic library).
- **fyp-llm-lib (vendored):** A custom C++ wrapper around llama.cpp, providing LoadModel(), UnloadModel(), and Generate() APIs. Located at external/fyp-llm-lib/llm/LLMWrapper.h. This library is vendored as a Git submodule.
- **llama.cpp (transitive):** The underlying C library for LLM inference, used internally by LLMWrapper. Supports GGUF model format files.

Scope and Non-Goals

In Scope (Current / Near-Term)

- Core gameplay loop (movement, aiming, combat, basic enemies with AI).
- Room pipeline: RoomPlan data → room generators (CombatRoom, SpawnRoom, PortalRoom) → RoomInstance runtime build with wall collisions.
- Floor pipeline: deterministic floor graph generation → BFS-style 2D grid embedding → door placement from graph connectivity → corridor generation.
- Multi-floor dungeon progression with portal-trigger-driven floor advancement.
- Hub world with shop zones, NPC interactions, quest board, and upgrade/economy system.
- Weapon system with multiple archetypes (Pistol, Assault Rifle, Shotgun, Knife, Sword, Axe) and a 3-slot loadout model.
- Quest system with kill and fetch quest types, progress tracking, and reward payout.
- LLM integration for room descriptions, NPC replies, and quest metadata generation.
- Player HUD and menu UI (main menu, pause, game over, quest board).
- Iterating on collision correctness, response, and filtering.

Out of Scope

- Full physics engine (broadphase spatial partitioning, continuous collision detection, complex response).
- Networking or multiplayer functionality.
- Fully deterministic per-room decoration (acceptable to be random per run for prototype).
- Final UI/UX polish and accessibility features.
- Mobile or console platform targets.

Intentional Simplifications

Several deliberate simplifications were made to maintain development velocity and focus on core systems:

- Avoid over-abstracting early: The prototype keeps some logic in Game::update() while validating rules before extraction into dedicated classes.
- Avoid designing a "final" Entity Component System prematurely: The current OOP hierarchy (Enemy → GruntEnemy / TurretEnemy) is sufficient for the current entity count and is easier to reason about.
- Collision response uses simple position rollback rather than sliding or impulse-based resolution.
- The Game class acts as a centralised orchestrator — acceptable for a prototype of this scale but acknowledged as a refactor target if systems grow significantly.

Software Architecture

Architectural Style and Rationale

The system follows a centralised orchestrator architecture where the Game class acts as the primary integration point. The trade-off is that the Game class accumulates responsibilities as features are added, creating a recognised refactor target for later development.

Within subsystems, the project uses standard object-oriented patterns:

- Inheritance hierarchies for polymorphic entity types (Enemy → GruntEnemy, TurretEnemy; Projectile → NormalBulletProjectile).
- Interface-based design for collision participants (ICollidable) and room generators (IRoomGenerator).
- Strategy pattern for steering behaviours (ISteeringBehaviour → SeekBehaviour, ArriveBehaviour).
- Service/wrapper pattern for LLM integration (LLMService wraps LLMWrapper with async lifecycle management).
- Data-driven generation using blueprint structs (RoomPlan, FloorPlan, DungeonPlan) that separate data from runtime representation.

High-Level System Diagram

The following text-based diagram illustrates the high-level architecture and data flow between major subsystems:

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Game (Orchestrator)                         │
│                                                                      │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐             │
│  │  MenuUI  │  │PlayerHUD │  │QuestMgr  │  │LLMService│             │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘             │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐        │
│  │                  Runtime Entity Containers               │        │
│  │  Player │ Enemies[] │ Projectiles[] │ Triggers[]          │        │
│  │  Pickups[] │ Chests[] │ NPCs[]                           │        │
│  └──────────────────────────────────────────────────────────┘        │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐        │
│  │             Generation / Level Pipeline                  │        │
│  │  DungeonPlan → FloorGenerator → FloorLayoutGenerator     │        │
│  │  IRoomGenerator (CombatRoom/SpawnRoom/PortalRoom)        │        │
│  │  RoomPlan → RoomInstance → StaticCollision               │        │
│  │  RoomDoorUtils │ Corridor Builder                        │        │
│  └──────────────────────────────────────────────────────────┘        │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐        │
│  │                  Collision System                        │        │
│  │  ICollidable │ CollisionLayer/Profile │ CollisionCheck   │        │
│  └──────────────────────────────────────────────────────────┘        │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐        │
│  │                  Input System                            │        │
│  │  InputManager → Gamepad (XInput wrapper)                 │        │
│  └──────────────────────────────────────────────────────────┘        │
└──────────────────────────────────────────────────────────────────────┘
```

Major Subsystem Overview

Game Loop / Orchestration: Game owns the SFML window and drives update + render. It manages runtime mode (HUB / DUNGEON), transitions between modes, and acts as the integration point for cross-system interactions (collisions, room triggers, combat waves, floor transitions, hub interactions, LLM polling).

Input System: InputManager provides a static access point. Gamepad wraps XInput, normalising sticks to [-1,1], triggers to [0,1], and exposing button states (down/pressed/released) and rumble control.

Player: Player handles movement, aiming (mouse + right stick), weapon switching, and attack emission. It implements ICollidable and exposes hooks for upgrades, loadout management, and pickup effects.

Enemy / AI: Enemy base class provides shared rendering, health, and collision. GruntEnemy uses steering (Seek/Arrive) with distance hysteresis and melee attacks. TurretEnemy is stationary with projectile attacks on randomised timers.

Combat / Projectiles / Damage Triggers: Weapons gate fire by cooldown and emit either Projectiles (ranged) or DamageTriggers (melee). Game owns runtime containers and handles lifecycle, collision, and cleanup.

Collision: Layer/mask bitmask filtering with AABB intersection. ICollidable interface is implemented by all collidable entities. Response is centralised in Game::CollisionChecks().

Room / Floor / Dungeon Generation: Three-tier procedural pipeline: DungeonPlan → FloorPlan (graph) → RoomPlans (blueprints) → RoomInstances (runtime). Floor graphs are deterministic (seeded). Room details use rand() for variety.

LLM Integration: LLMService wraps LLMWrapper with async init/request/poll lifecycle. Game adds a job queue layer (LLMJobContext) for room/NPC/quest prompts, submitting one job at a time when idle.

UI / HUD: PlayerHUD renders health/ammo/coin/quest overlay. MenuUI manages screen flow (main menu, pause, gameplay, game over, quest board) with keyboard and controller navigation.

Quest System: QuestManager maintains a 3-slot quest board with kill and fetch quest types. Progress is tracked during dungeon runs and rewards are committed on return to hub.

Hub Economy / Upgrades: Game owns currency and upgrade state. Hub shop zones use world-space rectangle overlap checks. Shops support weapon upgrades, ammo upgrades, player stat upgrades, cosmetic cycling, and armory weapon swapping.

Ownership and Data Flow

The Game class is the sole owner of all major runtime state. This centralised ownership model simplifies lifecycle management and makes data flow explicit:

- Player (single instance)
- PlayerHUD, MenuUI (single instances)
- QuestManager (single instance)

- Enemies: std::vector<std::unique_ptr<Enemy>>
- Projectiles: std::vector<std::unique_ptr<Projectile>>
- Damage triggers: std::vector<std::unique_ptr<DamageTrigger>>
- Pickups: std::vector<std::unique_ptr<Pickup>>
- Chests: std::vector<std::unique_ptr<Chest>>
- Hub NPCs: std::vector<NPC>
- Room plans + room instances + room world positions
- Floor plan + floor layout
- Dungeon plan (seed, floor progression, completion)
- LLM service + job queue
- Economy/upgrade state (coins, weapon levels, player stat levels)

Data flow follows a clear sequence each frame: Input → Update entities → Resolve collisions → Manage combat waves → Handle floor transitions → Poll LLM → Process hub interactions → Update HUD → Render.

Class Design and Specifications

Core Class Hierarchy (UML Class Diagram — Textual Representation)

The following textual UML class diagram captures the primary inheritance and composition relationships in the system. This representation follows standard UML notation conventions adapted for a text-based format.

```
┌──────────────────────────────────────────────────────────────┐
│                        «interface»                           │
│                        ICollidable                           │
├──────────────────────────────────────────────────────────────┤
│ + getCollisionBounds()  : FloatRect        «pure virtual»    │
│ + getCollisionProfile() : CollisionProfile «pure virtual»    │
└──────────────────────────────────────────────────────────────┘
                          │ implements
        ┌─────────────────┴─────────────────┬─────────────────┐
        ▼                 ▼                 ▼                 ▼
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│   Player     │  │    Enemy     │  │  Projectile  │  │DamageTrig.   │
├──────────────┤  │  «abstract»  │  │  «abstract»  │  ├──────────────┤
│ p_body       │  │ e_body       │  │ p_body       │  │dt_lifetime   │
│ p_health     │  │ e_health     │  │ p_damage     │  │dt_damage     │
│ p_weapons    │  │ e_target     │  │ p_speed      │  └──────────────┘
├──────────────┤  ├──────────────┤  ├──────────────┤
│ update()     │  │ update()=0   │  │ update()=0   │
│ hitWall()    │  │ hitWall()=0  │  │ destroy()    │
└──────────────┘  │ setTarget()  │  └──────────────┘
                  └──────────────┘          │
                        │                   ▼
        ┌───────────────┴───────────────┐  ┌──────────────────┐
        ▼                               ▼  │NormalBulletProj. │
┌──────────────┐  ┌──────────────┐       ├──────────────────┤
│GruntEnemy    │  │TurretEn.     │       │ init()           │
├──────────────┤  ├──────────────┤       │ update()         │
│e_agent       │  │te_weapon     │       └──────────────────┘
│e_seek        │  │te_timer      │
│e_arrive      │  └──────────────┘
│ge_weapon     │
└──────────────┘
```

Additional ICollidable implementors: Pickup, Chest, NPC, StaticCollision

```
┌──────────────────────┐        ┌──────────────────────┐
│ «interface»          │        │     SteeringAgent    │
│ ISteeringBehaviour   │        ├──────────────────────┤
├──────────────────────┤        │ position, velocity   │
│ getSteering()=0      │        │ maxSpeed, maxAccel.  │
└──────────────────────┘        └──────────────────────┘
          │
    ┌─────┴─────┐
    ▼           ▼
┌──────────┐  ┌──────────┐
│  Seek    │  │ Arrive   │
│Behav.    │  │ Behav.   │
└──────────┘  └──────────┘

┌──────────────────────┐        ┌──────────────────────┐
│ «interface»          │        │   Weapon             │
│ IRoomGenerator       │        ├──────────────────────┤
├──────────────────────┤        │ w_type, w_level      │
│ generateRoomPlan()   │        │ w_fireRate, w_cooldown │
└──────────────────────┘        ├──────────────────────┤
          │                     │ fire() [ranged/melee] │
    ┌─────┴─────┬─────┐         │ spawnProjectile()=0  │
    ▼           ▼     ▼         │ spawnMeleeHit()=0    │
┌──────┐  ┌──────┐  ┌──────┐    └──────────────────────┘
│Combat│  │Spawn │  │Portal│
│Room  │  │Room  │  │Room  │
└──────┘  └──────┘  └──────┘
```

ICollidable Interface

ICollidable is the central interface for the collision system. Any object that participates in collision checks must implement this interface. It requires two pure virtual methods:

- getCollisionBounds() → sf::FloatRect: Returns the axis-aligned bounding box of the object.
- getCollisionProfile() → CollisionProfile: Returns the object's collision layer and mask.

The CollisionProfile struct contains a CollisionLayer (bitmask flag identifying what the object is) and a uint32_t mask (identifying what layers the object can collide with). The canCollideWith() method performs bidirectional mask checking: A.mask & B.layer AND B.mask & A.layer must both be non-zero.

Current ICollidable implementors: Player, Enemy (and derivatives), Projectile (and derivatives), DamageTrigger, Pickup, Chest, NPC, and StaticCollision.

Enemy Hierarchy

The Enemy class is an abstract base providing shared body setup/rendering, target tracking, collision profile, health/death state, and damage handling. Derived classes must implement init(), update(float dt), setTarget(), hitWall(), and getEnemyType().

GruntEnemy: A steering-driven melee attacker. It owns a SteeringAgent and switches between SeekBehaviour (when far from target) and ArriveBehaviour (when close) using distance hysteresis thresholds. Within arrive range, it uses an EnemyWeapon (KNIFE) to spawn DamageTrigger instances for melee attacks.

TurretEnemy: A stationary ranged attacker. It tracks the player's direction and fires projectiles using an EnemyWeapon (PISTOL) on a randomised cooldown timer. It does not move but implements hitWall() for consistency with the base class contract.

Steering Behaviours (Strategy Pattern)

The steering system follows Reynolds' (1999) model and implements the Strategy design pattern. ISteeringBehaviour is an abstract interface with a single pure virtual method: getSteering(const SteeringAgent& agent) → Vector2f.

SteeringAgent is a lightweight struct holding position, velocity, maxSpeed, and maxAcceleration. The agent is updated each frame by applying the steering output as acceleration.

SeekBehaviour: Calculates a desired velocity directly toward the target at maximum speed, then returns the steering vector (desired - current velocity), clamped to maxAcceleration.

ArriveBehaviour: Similar to Seek but with a deceleration zone. When the agent is within the slowRadius, the desired speed is scaled proportionally to distance, producing a smooth deceleration to the target.

Projectile Hierarchy

Projectile is an abstract base class providing body rendering, lifetime tracking, collision support (via ICollidable), damage output, and destruction signalling. The collision profile depends on the source: player-fired bullets use PLAYER_BULLET_LAYER (masking

ENEMY_LAYER, WALL_LAYER, DOOR_LAYER), while enemy-fired bullets use ENEMY_BULLET_LAYER (masking PLAYER_LAYER, WALL_LAYER, DOOR_LAYER).

NormalBulletProjectile: The current concrete bullet type. It normalises direction on init, moves by direction × speed × dt each frame, and expires by either lifetime expiration or collision destruction.

Weapon System

The Weapon class supports both ranged and melee weapon archetypes. It gates fire by cooldown and delegates projectile/trigger creation to pure virtual methods (spawnProjectile, spawnMeleeHit). WeaponType enum currently includes: PISTOL, ASSAULT_RIFLE, SHOTGUN (ranged); KNIFE, SWORD, AXE (melee). Each weapon type configures its own fire rate, damage, fire mode (AUTOMATIC/BURST), and rumble intensity.

Player weapons use a loadout model (WeaponInLoadout) with a maximum of 3 equipped slots. Weapons are rebuilt from loadout entries via buildWeaponsFromLoadout(). Weapon level scaling adjusts damage and fire rate tuning. Ranged attacks consume ammo from the player's shared ammo pool, with shotgun consuming more ammo per attack than other ranged weapons.

Room Generation Pipeline

The room generation pipeline separates data from runtime representation using a two-stage workflow:

Stage 1 — Blueprint Generation: IRoomGenerator implementations (CombatRoom, SpawnRoom, PortalRoom) produce RoomPlan structs. A RoomPlan contains metadata (id, type, seed), dimensions (width, height, tileSize), a 1D tile map (indexed as row × width + column), and vectors of DoorPlan, SpawnerPlan, and TriggerPlan entries. CombatRoom randomises width (15–20) and height (10–13), places outer walls, random interior obstacles, and spawner tiles proportional to interior area. SpawnRoom and PortalRoom use fixed sizes (11×11).

Stage 2 — Runtime Build: RoomInstance.buildFromPlan() converts a RoomPlan into renderable SFML shapes and StaticCollision colliders. Wall tiles become wall shapes + WALL_LAYER colliders. Door tiles become door shapes with optional DOOR_LAYER colliders (when locked). Trigger plans become colliders with appropriate layer flags (PORTAL_TRIGGER_LAYER, DOOR_TRIGGER_LAYER).

CRC Cards

The following Class-Responsibility-Collaborator (CRC) cards summarise the primary responsibilities and collaborations for key classes in the system:

CRC Card: Game

Responsibilities	Collaborators
Own SFML window and drive game loop	Player
Manage game mode (HUB/DUNGEON) and transitions	Enemy
Update and render all subsystems	Projectile

Centralise collision checks	DamageTrigger
Manage combat waves and floor transitions	Pickup
Route menu actions and hub interactions	Chest
Poll LLM service and dispatch results	NPC
Manage economy/upgrade state	MenuUI
	PlayerHUD
	QuestManager
	LLMService
	RoomInstance
	FloorGenerator
	DungeonPlan

CRC Card: Player

Responsibilities	Collaborators
Handle movement input (keyboard/gamepad)	InputManager
Handle aiming input (mouse/right stick)	Weapon
Manage weapon loadout and switching	Projectile
Emit projectiles and damage triggers	DamageTrigger
Track health/ammo state	ICollidable
Respond to wall collisions	
Apply upgrades and pickup effects	

CRC Card: Enemy (abstract)

Responsibilities	Collaborators
Provide shared body setup and rendering	ICollidable
Track target position	CollisionProfile
Manage health/death state	
Implement ICollidable interface	

CRC Card: GruntEnemy

Responsibilities	Collaborators
Steer toward player using Seek/Arrive	SteeringAgent
Switch behaviours by distance hysteresis	SeekBehaviour
Attack with melee weapon (KNIFE)	ArriveBehaviour
	EnemyWeapon
	DamageTrigger

CRC Card: TurretEnemy

Responsibilities	Collaborators
Track player direction	EnemyWeapon
Fire projectiles on randomised timer	Projectile

CRC Card: FloorGenerator

Responsibilities	Collaborators

Generate deterministic floor graph from seed	FloorPlan
Enforce room type constraints (spawn first, portal last)	FloorNode
Prevent direct spawn-portal edges	FloorEdge
	std::mt19937

CRC Card: RoomInstance

Responsibilities	Collaborators
Build SFML shapes from RoomPlan	RoomPlan
Build StaticCollision colliders for walls/doors/triggers	StaticCollision
Render room geometry	

CRC Card: LLMService

Responsibilities	Collaborators
Manage async model init/shutdown	LLMWrapper
Queue generation requests on worker thread	std::thread
Poll for completed responses	std::mutex
Report readiness/busy state	std::atomic

CRC Card: QuestManager

Responsibilities	Collaborators
Maintain 3-slot quest board	QuestData
Generate kill and fetch quests	EnemyType
Track active quest progress	PickupType
Finalise run results and commit rewards	

CRC Card: CollisionCheck

Responsibilities	Collaborators
Perform AABB intersection with layer/mask filtering	ICollidable
	CollisionProfile

Algorithm Design

Floor Graph Generation Algorithm

The FloorGenerator produces a deterministic floor graph (FloorPlan) from a seed. The algorithm ensures that the floor is always solvable and follows structural constraints.

```
FUNCTION generateFloorPlan(floorId, dungeonSeed, isBossFloor):
    floorSeed ← dungeonSeed + floorId
    rng ← mt19937(floorSeed)

    roomCount ← randomInt(rng, 3, 10)

    // Create room nodes
    rooms[0] ← {id: 0, type: SPAWN}
    FOR i = 1 TO roomCount - 2:
        rooms[i] ← {id: i, type: COMBAT}
    rooms[roomCount-1] ← {id: roomCount-1, type: PORTAL}

    // Generate edges (connections)
    edges ← []
    FOR i = 1 TO roomCount - 1:
        // Connect each room to a random earlier room
        parent ← randomInt(rng, 0, i - 1)

        // Prevent direct spawn → portal edge
        IF parent == 0 AND i == roomCount - 1:
            IF roomCount > 2:
                parent ← randomInt(rng, 1, i - 1)

        edges.add({id_a: parent, id_b: i})

    // Build adjacency list
    plan.buildAdjacencyList()

    RETURN plan
```

Analysis: The algorithm guarantees connectivity (every room is reachable from spawn) by construction — each new room connects to an existing room. The spawn-portal constraint prevents trivially short paths. Time complexity is O(n) where n is the room count. The use of std::mt19937 ensures deterministic output for a given seed, enabling reproducible floor layouts for testing and seeded runs.

Floor Layout Embedding Algorithm

FloorLayoutGenerator embeds the floor graph into a 2D grid, producing world-space positions for each room. The algorithm uses a BFS traversal to place rooms without overlaps.

```
FUNCTION generateFloorLayout(floorPlan, roomPlans):
    grid ← empty 2D occupied-cells map
    positions ← array of Vector2f[roomCount]

    // Place spawn room at origin
    positions[0] ← (0, 0)
    markOccupied(grid, room[0], (0, 0))
```

```
    queue ← BFS queue starting from room 0

    WHILE queue is not empty:
        currentId ← queue.dequeue()

        FOR EACH neighbour IN adjacencyList[currentId]:
            IF neighbour is already placed:
                CONTINUE

            // Try directions: EAST, SOUTH, WEST, NORTH
            FOR EACH direction IN {E, S, W, N}:
                candidatePos ← positions[currentId] + offset(direction, roomSize)

                IF NOT overlaps(grid, room[neighbour], candidatePos):
                    positions[neighbour] ← candidatePos
                    markOccupied(grid, room[neighbour], candidatePos)
                    queue.enqueue(neighbour)
                    BREAK

    RETURN positions
```

Analysis: The BFS traversal ensures that rooms are placed in proximity order from the spawn room, producing compact layouts. The overlap check prevents room collisions. In the worst case, if all preferred directions are occupied, the algorithm may need to search further, but for typical floor sizes (3–10 rooms) this is not a practical concern.

Corridor Generation Algorithm

Corridors are generated to physically connect rooms that share a floor graph edge. Each corridor is built as a temporary RoomPlan of type CORRIDOR and immediately converted to a RoomInstance.

```
FUNCTION buildCorridors(floorPlan, roomPlans, roomWorldPositions):
    FOR EACH edge IN floorPlan.edges:
        roomA ← roomPlans[edge.id_a]
        roomB ← roomPlans[edge.id_b]

        // Find matching door pairs (one door on each side)
        doorA ← findDoorFacing(roomA, directionOf(roomA → roomB))
        doorB ← findDoorFacing(roomB, directionOf(roomB → roomA))

        // Calculate world-space door positions
        startPos ← worldPosition(roomA) + tileToWorld(doorA)
        endPos   ← worldPosition(roomB) + tileToWorld(doorB)

        // Build corridor plan: a narrow strip of floor tiles
        //   bordered by walls on both sides
        corridorPlan ← buildCorridorStrip(startPos, endPos, tileSize)

        // Convert to runtime instance
        corridorInstance ← new RoomInstance()
        corridorInstance.buildFromPlan(corridorPlan, corridorWorldPos)

        appendToRoomInstances(corridorInstance)
```

Collision Detection and Resolution

The collision system uses a two-phase approach: layer/mask filtering followed by AABB intersection. This is evaluated for all relevant entity pairs each frame in Game::CollisionChecks().

```
FUNCTION areColliding(entityA: ICollidable, entityB: ICollidable) → bool:
    profileA ← entityA.getCollisionProfile()
    profileB ← entityB.getCollisionProfile()

    // Phase 1: Layer/mask filtering
    IF (profileA.mask & profileB.layer) == 0:
        RETURN false
    IF (profileB.mask & profileA.layer) == 0:
        RETURN false

    // Phase 2: AABB intersection
    boundsA ← entityA.getCollisionBounds()
    boundsB ← entityB.getCollisionBounds()

    RETURN boundsA.intersects(boundsB)

FUNCTION CollisionChecks():
    // Player vs enemies
    FOR EACH enemy IN enemies:
        IF areColliding(player, enemy):
            player.takeDamage(contactDamage)

    // Projectiles vs enemies/player
    FOR EACH projectile IN projectiles:
        FOR EACH enemy IN enemies:
            IF areColliding(projectile, enemy):
                enemy.takeDamage(projectile.applyDamage())
                projectile.destroy()
        IF areColliding(projectile, player):
            player.takeDamage(projectile.applyDamage())
            projectile.destroy()

    // Entities vs walls/doors (rollback response)
    FOR EACH wall IN staticColliders:
        IF areColliding(player, wall):
            player.hitWall()  // restore previous position
        FOR EACH enemy IN enemies:
            IF areColliding(enemy, wall):
                enemy.hitWall()
        FOR EACH projectile IN projectiles:
            IF areColliding(projectile, wall):
                projectile.destroy()

    // Pickups/Chests vs player
    FOR EACH pickup IN pickups:
        IF areColliding(player, pickup):
            applyPickupEffect(player, pickup)
            pickup.destroy()

    // Portal/Door triggers
    FOR EACH trigger IN triggers:
        IF areColliding(player, trigger):
            handleTriggerInteraction(trigger)
```

Analysis: The current approach is $O(n^2)$ for pairwise checks. This is acceptable for the prototype's typical entity counts (< 50 entities per room). If entity counts scale significantly, a broadphase spatial partitioning structure (e.g., spatial hash grid or quadtree) would be required to maintain performance.

Combat Wave Management

Combat rooms support a wave-based spawning system. When the player enters an uncleared combat room via a door trigger, combat begins:

```
FUNCTION onDoorTrigger(roomId):
    IF roomPlans[roomId].type == COMBAT AND NOT roomPlans[roomId].isCleared:
        startCombat(roomId)

FUNCTION startCombat(roomId):
    m_isInCombat ← true
    m_waveCounter ← totalWaves  // typically 2-3 based on room size
    lockDoors(roomId)
    generateNewWave(roomPlans[roomId])
    spawnEnemies(roomId)

FUNCTION ManageWave():
    IF NOT m_isInCombat:
        RETURN

    IF enemies.isEmpty():
        m_waveCounter ← m_waveCounter - 1

        IF m_waveCounter <= 0:
            // Room cleared
            m_isInCombat ← false
            unlockDoors(m_activeRoomId)
            roomPlans[m_activeRoomId].isCleared ← true
        ELSE:
            // Next wave
            generateNewWave(roomPlans[m_activeRoomId])
            spawnEnemies(m_activeRoomId)
```

LLM Asynchronous Job Queue

The LLM integration uses an asynchronous architecture to prevent blocking the game thread. The LLMService manages worker threads for both model initialisation and text generation. The Game class adds a queue layer to serialise multiple LLM requests.

```
// LLMService async lifecycle
FUNCTION initAsync(modelPath):
    IF already initialising: RETURN false
    llm_initInProgress ← true
    SPAWN thread: initWorkerMain(modelPath)
    RETURN true

FUNCTION initWorkerMain(modelPath):
    success ← llm_wrapper.LoadModel(modelPath)
    LOCK(llm_initMutex):
        llm_initResult ← success
        llm_isReady ← success
    llm_initInProgress ← false

FUNCTION requestGenerate(prompt):
    IF NOT ready OR busy: RETURN false
    llm_isBusy ← true
    SPAWN thread: generateWorkerMain(prompt)
    RETURN true

FUNCTION generateWorkerMain(prompt):
    response ← llm_wrapper.Generate(prompt)
```

```
    LOCK(llm_resultMutex):
        llm_latestResponse ← response
    llm_isBusy ← false

// Game job queue layer
FUNCTION processLLMQueue():
    IF llmService.isBusy() OR llmJobQueue.empty():
        RETURN

    job ← llmJobQueue.front()
    llmJobQueue.pop()
    m_currentLLMJob ← job
    m_hasActiveLLMJob ← true
    llmService.requestGenerate(job.prompt)

FUNCTION update(dt):
    // ... other updates ...

    // Poll init completion
    initResult ← llmService.tryConsumeInitResult()
    IF initResult.has_value():
        IF initResult == true:
            queueQuestMetadataJobs()

    // Poll generation completion
    response ← llmService.tryConsumeLatestResponse()
    IF response.has_value():
        handleLLMResponse(response.value())
        m_hasActiveLLMJob ← false

    processLLMQueue()
```

Analysis: This architecture ensures the game thread never blocks on LLM operations. The queue serialises requests to prevent concurrent model access. Thread safety is achieved through std::atomic flags and std::mutex-protected shared data. The tryConsume pattern (optional return) allows non-blocking polling.

Runtime Flow

Per-Frame Update Sequence

The game uses a fixed-timestep update loop. The following sequence describes the operations performed each frame during gameplay:

Step	Description
1. Event Processing	Poll SFML window events. Route through game event handling: if a menu is active, process menu input; otherwise process gameplay key events.
2. Input Reading	InputManager::update() reads current keyboard/mouse/gamepad state.
3. Controller Handling	ControllerInputHandler() processes pause/menu toggling, camera toggle, and context-dependent interactions.
4. Menu Guard	If the menu is not on the gameplay screen, return early from update.
5. Aim Input Preparation	Select the active gameplay view (player camera or floor overview). Map mouse pixel position into world-space for aiming.
6. Death Check	If the player is dead in dungeon mode: finalise quest run state, set menu to GAME_OVER, return from update.
7. Active Room Update	updateActiveRoom() sets m_activeRoomId based on which room's bounds contain the player.
8. Player Update	Update movement, aiming, weapon logic (may emit projectiles or melee triggers).
9. Enemy Update	Set targets to player position. Run type-specific updates (TurretEnemy projectile path, GruntEnemy melee path). On death: record quest kill progress, roll pickup spawn, remove enemy.
10. Transient Object Cleanup	Update/remove destroyed projectiles and expired damage triggers.
11. Collision Resolution	CollisionChecks() resolves all entity pair interactions (see Section 6.4).
12. Pickup Cleanup	Remove destroyed pickups from the runtime list.
13. Combat Wave Management	ManageWave() handles wave progression (see Section 6.5).
14. Floor Progression	Consume m_requestNextFloor flag. Advance dungeon plan. Load next floor or return to hub.
15. LLM Polling	Consume async init results. Consume latest LLM response. Process LLM job queue.
16. Hub Interactions	When in HUB mode: process shop purchases, job board opening, NPC dialogue.
17. HUD Update	Update PlayerHUD values from player/economy/quest state.
18. Render	Choose camera mode. Render rooms → player → enemies → projectiles → triggers → pickups → chests → NPCs → overlays. Switch to default view. Render HUD or menu UI.

State Transitions (Game Modes)

The game operates in two primary modes with defined transitions:

```
Game Start → ┌──────────────┐
             │  MAIN MENU   │
             └──────────────┘
                    │ "New Game" / "Continue"
             ┌──────▼───────┐
             │   HUB WORLD  │ ◄── Return from dungeon
             │              │ ◄── Game Over → Hub
             └──────────────┘
                    │ Portal interaction
             ┌──────▼───────┐
             │   DUNGEON    │
             │ (per floor)  │ ── Portal → Load next floor
             └──────────────┘              (loop until final)
                    │
          ┌─────────┴─────────┐
          │                   │
     Player dies       Final floor complete
          │                   │
     ┌────▼────┐         ┌────▼──────┐
     │GAME OVER│         │ Return to │
     │ screen  │         │ HUB (reward)│
     └─────────┘         └───────────┘
```

Data and State Management

Runtime State Ownership

Runtime state is centrally owned by the Game class. The following table summarises the key state categories, their owners, and their types:

State	Owner	Type / Notes
Game mode	Game	GameMode enum (HUB / DUNGEON)
Menu state	Game	MenuUI with screen enum, button selections, pending action
Player entity	Game	Player instance with transform, velocity, health, ammo, weapons
Enemy list	Game	std::vector<std::unique_ptr<Enemy>>
Projectile list	Game	std::vector<std::unique_ptr<Projectile>>
Damage triggers	Game	std::vector<std::unique_ptr<DamageTrigger>>
Pickups	Game	std::vector<std::unique_ptr<Pickup>>
Chests	Game	std::vector<std::unique_ptr<Chest>>
Hub NPCs	Game	std::vector<NPC> with dialogue state, active NPC tracking
Room plans	Game	std::vector<RoomPlan> — blueprint data per room
Room instances	Game	std::vector<RoomInstance> — runtime shapes and colliders
Floor plan	Game	FloorPlan struct (graph nodes + edges + adjacency)
Dungeon plan	Game	DungeonPlan struct (seed, floor id, completion)
Economy	Game	int m_coins + weapon/player upgrade level ints
Quest state	Game	QuestManager (board quests, active quest, run finalisation)
LLM state	Game	LLMService + job queue + presentation buffers
Camera state	Game	sf::View objects for player, floor, and game cameras
Player health	Player	int p_health, p_maxHealth, bool p_isDead
Player weapons	Player	std::vector<WeaponInLoadout> + runtime weapon instances
Enemy health	Enemy	int e_health, e_maxHealth, bool e_isDead (per instance)
Steering state	GruntEnemy	SteeringAgent (position, velocity, limits)

Procedural Generation State

The procedural generation system maintains a clear separation between deterministic and non-deterministic elements:

Deterministic (seeded): Floor graph structure, room connectivity, room count, room type assignment. Uses std::mt19937 seeded with dungeonSeed + floorId. The same seed always produces the same floor graph.

Non-deterministic (random): Room interior details (obstacle count, obstacle positions, spawner positions). Uses rand() for per-run variety. This is an intentional design decision: while the overall floor structure is reproducible, individual room appearances vary between runs of the same seed.

Dungeon progression: DungeonPlan tracks seed, currentFloorId, floorCount, and isDungeonComplete. The dungeon currently uses 3 floors per run with a fixed test seed.

Critical Analysis

Known Prototype Shortcuts

The following prototype shortcuts have been identified and documented. Each represents a conscious trade-off between development velocity and production readiness:

Collision response: Uses simple position rollback (restore previous position on collision) rather than sliding or impulse-based resolution. This can cause "sticking" against walls when moving diagonally. A sliding response would improve player feel.

Collision broadphase: All collision checks are brute-force pairwise comparisons ($O(n^2)$). Acceptable for current entity counts (<50 per room) but would require spatial partitioning (spatial hash grid or quadtree) for larger scenes.

Non-deterministic room details: Room obstacle/spawner placement uses rand() rather than seeded generators. While acceptable for gameplay variety, this means replay/deterministic testing of individual rooms is not currently possible.

Game class complexity: The Game class acts as a "god object" for orchestration. While this is a common and pragmatic approach for prototypes, it concentrates many responsibilities and would benefit from extraction of subsystems (e.g., CollisionManager, CombatManager, HubManager) as the project matures.

Active room detection: Simple bounds checking against room rectangles. May produce incorrect results if rooms overlap or if the player is in a corridor between rooms.

Corridor generation: Procedural corridor building may need additional edge-case handling for alignment and overlap situations.

LLM response handling: Text is consumed as complete responses (no streaming). Response validation is lightweight (simple tagged-line parsing for quest text). No formal latency budget policy exists.

Dungeon completion: Flow currently returns directly to hub mode without a dedicated run-results or end-of-run screen.

Technical Risks

Risk	Severity	Mitigation
Performance degradation	Medium	If entity counts scale beyond current levels without broadphase collision, frame rate will degrade. Mitigation: implement spatial hash grid when entity counts exceed ~100 per room.
Complexity growth	Medium	As room types, interactions, and combat systems expand, the centralised Game class will become increasingly difficult to maintain. Mitigation: extract subsystem managers progressively.
LLM latency	Low–Med	LLM generation takes ~5.5s in Release x64 and ~30s in Debug. Long generation times could create dead spots in gameplay. Mitigation: pre-queue prompts during loading; implement caching for repeated contexts; consider streaming output.
LLM integration stability	Low	Moving from debug triggers to production dialogue flow requires robust error handling and fallback templates. Mitigation: implement response validation with fallback defaults for malformed LLM output.
Corridor alignment	Low	Edge cases in corridor generation could produce misaligned or overlapping corridor geometry. Mitigation: add alignment constraints and overlap detection in corridor builder.

Evaluation Against Goals

This section evaluates the current prototype against the must-have goals defined in Section 2.1:

Goal	Status	Evidence / Notes
Playable core loop	Achieved	Player can move (WASD/gamepad), aim (mouse/right stick), and attack with multiple weapon types. Enemies spawn with consistent AI behaviours (Seek/Arrive for Grunts, ranged for Turrets). Collision and damage are functional with layer/mask filtering.
Procedural dungeon	Achieved	Floors are generated from seeds with 3–10 rooms per floor. Three room types (Spawn, Combat, Portal) are implemented. Multi-floor progression with portal triggers is functional.
Upgrade loop	Achieved	Hub world exists with shop zones for weapon upgrades, player stat upgrades, ammo upgrades, cosmetic cycling, and armory weapon swapping. Currency (coins) is earned during runs from enemy kills and pickups.
NPC interaction	Achieved	Hub NPCs can be interacted with via overlap + input. LLM integration provides async dialogue generation. Quest board NPCs are linked to the quest system.
Quest system	Achieved	QuestManager supports kill and fetch quest types with a 3-slot board. Progress tracking and reward payout on run completion are functional.
LLM integration	Achieved	Async model loading, room description generation, NPC reply generation, and quest metadata generation are all functional. Performance benchmarks documented.

Overall Assessment: All must-have goals have been achieved at a prototype level. The core gameplay loop is playable with functional combat, procedural generation, hub progression, NPC interaction, and LLM integration. The architecture is readable and modular at the subsystem level, though the Game class concentration is a recognised area for improvement. Performance is stable at the target resolution with current entity counts.

Conclusions

This Technical Design Document has presented the complete technical architecture, class design, algorithm specifications, and critical analysis of Project Rival — a 2D isometric dungeon crawler roguelike built in C++ with SFML.

The project demonstrates several significant technical achievements:

- A working procedural dungeon generation pipeline that produces playable, interconnected floor layouts from a single seed value, using graph-based generation with BFS embedding.
- A modular collision system with bitmask layer/mask filtering, supporting diverse entity types (players, enemies, projectiles, damage triggers, pickups, chests, NPCs, walls, doors, and triggers) through a shared ICollidable interface.
- An AI system using Reynolds' steering behaviours (Seek, Arrive) with a clean Strategy pattern implementation, enabling enemy movement that is both effective and extensible.
- A complete weapon system supporting six weapon archetypes across ranged and melee categories, with a 3-slot loadout model, level scaling, and distinct fire modes.
- A novel locally-hosted LLM integration for dynamic content generation (room descriptions, NPC dialogue, quest lore), implemented with an asynchronous job queue architecture that prevents blocking the game thread.
- A hub-and-dungeon meta-progression loop with economy, upgrades, quest board, and NPC interactions.

The critical analysis has identified several areas where the prototype could be improved:

- Collision response could be enhanced from simple rollback to sliding resolution for smoother player movement against walls.
- The Game class's centralised orchestration role should be progressively decomposed into dedicated subsystem managers as feature scope grows.
- Broadphase spatial partitioning should be implemented before entity counts scale significantly beyond current levels.
- LLM response validation should be strengthened with formal schema checking and fallback templates for malformed output.
- Room generation could be made fully deterministic by replacing rand() calls with per-room seeded generators.

The design decisions documented throughout this TDD reflect a prototype-first philosophy: build simple, working systems first, validate gameplay rules, then refactor toward production-quality architecture. This approach has proven effective for a Final Year Project, enabling rapid iteration while maintaining a codebase that is readable, testable, and extensible.

Future work would focus on implementing the stretch goals (additional room types, procedural boss generation, save/load persistence), hardening the LLM integration for production dialogue flows, and progressively decomposing the centralised Game class into dedicated subsystem managers.

References

Books

[1] Gregory, J. (2014). Game Engine Architecture. 2nd ed. Boca Raton: CRC Press.

[2] Millington, I. and Funge, J. (2009). Artificial Intelligence for Games. 2nd ed. Burlington: Morgan Kaufmann.

[3] Nystrom, R. (2014). Game Programming Patterns. Genever Benning.

[4] Ericson, C. (2004). Real-Time Collision Detection. San Francisco: Morgan Kaufmann.

Academic Papers and Reports

[5] Reynolds, C. W. (1999). Steering Behaviors For Autonomous Characters. Proceedings of Game Developers Conference 1999, San Jose, CA. Miller Freeman Game Group.

[6] Togelius, J., Yannakakis, G.N., Stanley, K.O. and Browne, C. (2011). Search-Based Procedural Content Generation: A Taxonomy and Survey. IEEE Transactions on Computational Intelligence and AI in Games, 3(3), pp. 172–186.

Web Resources

[7] SFML Team. (2024). SFML - Simple and Fast Multimedia Library. [Online]. (https://www.sfml-dev.org/). (Accessed 13 April 2026).

[8] Gerganov, G. (2024). llama.cpp — Inference of Meta's LLaMA model in pure C/C++. [Online]. (https://github.com/ggerganov/llama.cpp). (Accessed 13 April 2026).

[9] Microsoft. (2024). XInput Game Controller APIs. [Online]. (https://learn.microsoft.com/en-us/windows/win32/xinput/). (Accessed 13 April 2026).

[10] cppreference.com. (2024). std::mt19937 — Mersenne Twister pseudo-random generator. [Online]. (https://en.cppreference.com/w/cpp/numeric/random/mersenne_twister_engine). (13 April March 2026).